

Exercise Report

Geometric Numerical Integration

SS 2020

Georg Lukas Zikeli, 22020277

Submission: 19.10.2020

Contents

1	Analysis of the Newton-Raphson algorithm	2
1.1	From exercise 2a(ii)	2
1.2	From exercise 2b(ii)	3
2	Euler-Lagrange equations of two example systems	4
2.1	Planar pendulum, minimal coordinate: $\mathbf{q} = \varphi$	5
2.2	Kepler-problem, cartesian coordinates $\mathbf{q} = (x, y)$	6
3	Calculation of the discrete Euler-Lagrange equations for the Kepler problem	7
3.1	Variational integration based on the Midpoint Rule	7
3.2	Variational integration based on the Trapezoidal Rule	8
3.3	Post-Processing of results obtained from Variational Integrators	8
4	Visualization and interpretation of results based on different numerical methods for simulating the example systems	9
4.1	Euler methods	9
4.1.1	Explicit Euler algorithm	9
4.1.2	Implementation and evaluation of the Explicit Euler algorithm	10
4.1.3	Implicit Euler algorithm	12
4.1.4	Implementation and evaluation of the Implicit Euler algorithm	12
4.2	Symplectic integrators	13
4.2.1	Symplectic Euler method	13
4.2.2	Implementation and evaluation of the Symplectic Euler algorithm . . .	14
4.2.3	Störmer-Verlet method	14
4.2.4	Implementation and evaluation of the Störmer-Verlet algorithm as a one-step method	15
4.3	Runge-Kutta methods	16
4.3.1	The general Runge-Kutta method	17
4.3.2	Implementation of the general Runge-Kutta method	17
4.3.3	Evaluation of the general Runge-Kutta method by the RK4 method . .	18
4.3.4	Evaluation of the general Runge-Kutta method by the Radau-IIA method	18
4.4	Variational integrators evaluated by the Kepler problem	20
4.5	Comparison of numerical solutions with the analytical	20
4.6	Summary	21
5	Conserved quantities of the Kepler problem	22
5.1	Conservation of energy	22
5.2	Conservation of linear and angular momentum	24
5.3	Conservation of physical quantities by the VI-TP integrator in a general context	25

5.4	Summary	26
6	Numerical analysis on symmetry and order of convergence	27
6.1	Symmetry of numerical ODE solvers	27
6.1.1	Symmetry of the Stoermer-Verlet method	27
6.1.2	Symmetry of the implicit Euler algorithm	29
6.1.3	Symmetry of the classical Runge-Kutta method	30
6.2	Order of convergence	31
6.2.1	Order of convergence of the Explicit Euler algorithm	31
6.2.2	Order of convergence of the Stoermer-Verlet algorithm	32
6.2.3	Order of convergence of the RadauIIA algorithm	32

List of Figures

4.1	Simulation results for different values of h using the Explicit Euler algorithm	10
a	Planar pendulum	10
b	Kepler problem (Jupiter)	10
4.2	Simulation results using Explicit Euler with varying simulation parameters . .	11
a	Planar pendulum	11
b	Kepler problem (Neptune)	11
4.3	Neptune's phase space diagram of the Kepler problem for different step sizes .	11
4.4	Simulation results for different values of h using the Implicit Euler algorithm	12
a	Planar pendulum	12
b	Kepler problem (Jupiter)	12
4.5	Simulation results for different values of h using the Symplectic Euler algorithm	14
a	Planar pendulum	14
b	Kepler problem (Jupiter)	14
4.6	Closeups of the results for using the Symplectic Euler algorithm	14
a	Planar pendulum	14
b	Kepler problem (Jupiter)	14
4.7	Simulation results for different values of h using the Störmer-Verlet algorithm	15
a	Planar pendulum	15
b	Kepler problem (Jupiter)	15
4.8	Simulation results for the planar pendulum in phase space with $h = 0.1$ using both the Stoermer-Verlet and the Symplectic-Euler methods	16
a	Planar pendulum	16
b	Planar pendulum - closeup	16
4.9	Simulation results for different values of h using the RK4 algorithm	18
a	Planar pendulum	18
b	Kepler problem (Jupiter)	18
4.10	Simulation results for different values of h using the Radau-IIA algorithm . .	19
a	Planar pendulum	19
b	Kepler problem (Jupiter)	19
4.11	Visualization of results obtained from applying the pendulum system to the RK4-method as well as the Radau-IIA method for exemplary values of h . . .	19
4.12	Simulation results for different values of h using the Variational Integrator based on the Trapezoidal Rule (Jupiter)	20
4.13	Simulation results after computing the numerical solutions of the Euler methods in contrast to the analytical solution	21
a	Kepler problem (Neptune), $h = 100$	21
b	Kepler problem (Neptune), $h = 1$, close-up	21

5.1	Evolution of the energetical error for the Kepler problem calculated by using Euler methods, $h = 0.1$ (Jupiter). ERROR: y-label should be e_E (*)	22
5.2	Evolution of the energetical error for the Kepler problem calculated by using symplectic methods, $h = 0.1, T = 8e4$ (Jupiter)	23
a	Regular plot	23
b	Logarithmic plot	23
5.3	Energy error evolution for the Kepler problem calculated by Runge Kutta methods and the Stoermer Verlet method, $h = 10, T = 8e4$, Jupiter. ERROR (*)	23
5.4	Error evolution for the linear momentum for the Kepler problem calculated by Runge Kutta methods, $h = 1, T = 4e4$, Jupiter	24
5.5	Error evolution for the angular momentum of the Kepler problem calculated by using different methods, $h = 1, T = 4e4$ (Jupiter)	25
a	Logarithmic plot	25
b	Regular plot	25
5.6	Evolution of physical quantities for numerical computation using the VI-TR and RadauIIA methods	26
a	$h = 10, T = 8e4$	26
b	$h = 1, T = 4e4$	26
6.1	Subsequent application of the Störmer-Verlet algorithm and its adjoint, $h = 0.1$	28
a	Stoermer-Verlet solution	28
b	Adjoint solution	28
6.2	Subsequent application of the Störmer-Verlet forwards and backwards, $h = 1$	28
6.3	Subsequent application of the implicit Euler algorithm and its analytical and numerical adjoint, $h = 0.01$	29
a	Hybrid simulation	29
b	Numerical simulation	29
6.4	Subsequent application of the classical Runge-Kutta algorithm and its numerical adjoint, $h = 0.1$	30
6.5	Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005, 0.001, 0.0005]$ after simulating with the Explicit Euler algorithm	31
6.6	Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005, 0.001, 0.0005]$ after simulating with the Stoermer-Verlet algorithm	32
6.7	Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005]$ after simulating with the RadauIIA algorithm	33

List of Tables

1.1	Value and error for each iteration until ε_{min} was met	2
1.2	Value and error for each iteration until ε_{min} was met	3
5.1	Qualitative conclusion on conservation of properties	26
6.1	Numerical adjoint errors for the Stoermer-Verlet method for different h	28
6.2	Numerical adjoint errors for the Implicit Euler method for different h	29
6.3	Numerical adjoint errors for the Classical Runge-Kutta method for different h	30

Preamble:

This report presents the computational results and some basic investigative analysis on the latter for tasks of the course *Geometric Numerical Integration*. Algorithms representing the underlying numerical methods and simulation control were implemented and executed using `Octave` version 4.2.2 on `Ubuntu 18.04 LTS`.

Plots have been rendered using `fltk` version 1.3.5.

All functions, scripts, datasets and plots can be provided upon request.

Sadly, I have made some mistakes regarding the labeling of axis, especially in plots showing errors in energy and angular momentum. As time is getting short, I will leave them the way they are and appologize hereby.

1 Analysis of the Newton-Raphson algorithm

The Newton-Raphson algorithm is of great use for solving implicit steps of numerical integration methods, which is why its implementation and an exemplary evaluation make a good starting point for our investigation. For $F \in C^1(\mathbb{R}^n, \mathbb{R}^n)$ and its derivative $DF \in C^0(\mathbb{R}^n, \mathbb{R}^n)$ as well as an initial value $x_0 \in \mathbb{R}^n$ the general formula for the algorithm is given as

$$x_{n+1} = x_n + (DF(x_n))^{-1}(y - F(x_n)) \quad (1.1)$$

The order of convergence of the iteration in each step was calculated by

$$\mu(n) = \frac{\log(\varepsilon_n)}{\log(\varepsilon_{n-1})}, \text{ with } \varepsilon_n = |f(x_i)| \quad (1.2)$$

Results of applying this algorithm to different functions in order to solve for their approximate roots or, respectively, minima with a minimum precision of $\varepsilon_{min} = 10^{-12}$ are shown in the tables below within the subsections.

1.1 From exercise 2a(ii)

- Function: $g(y) = 10y^3 - 9y + 2$
- Derrivative: $g'(y) = 30y^2 - 9$
- Starting Value: $y_0 = -1$

Table 1.1: Value and error for each iteration until ε_{min} was met

iteration	value	error	order of convergence
1	$-1.04761e^{01}$	$6.91070e^{-02}$	$0.00000e^{01}$
2	$-1.04473e^{01}$	$2.61974e^{-04}$	$3.08644e^{01}$
3	$-1.04472e^{01}$	$3.81540e^{-09}$	$2.35038e^{01}$
4	$-1.04472e^{01}$	$1.77636e^{-15}$	$1.75216e^{01}$

Here, the algorithm was used in order to solve for the roots of $g(y)$. The order of convergence implies that the iteration converges quadratically.

1.2 From exercise 2b(ii)

- Function: $l(u, v) = u \exp(-u^2 - v^2)$
- Jacobian: $l'(u, v) = \begin{bmatrix} \frac{\partial l}{\partial u} \\ \frac{\partial l}{\partial v} \end{bmatrix} = \begin{bmatrix} (1 - 2u^2) \exp(-u^2 - v^2) \\ -2uv \exp(-u^2 - v^2) \end{bmatrix} := J$
- Hessian: $H := \begin{bmatrix} \frac{\partial J_1}{\partial u} & \frac{\partial J_1}{\partial v} \\ \frac{\partial J_2}{\partial u} & \frac{\partial J_2}{\partial v} \end{bmatrix} = \begin{bmatrix} (4u^3 - 6u) \exp(-u^2 - v^2) & (-2v + 4u^2v) \exp(-u^2 - v^2) \\ (-2v + 4u^2v) \exp(-u^2 - v^2) & (-2u + 4uv^2) \exp(-u^2 - v^2) \end{bmatrix}$
- Starting Value: $y_0 = [-0.5, 0.1]$

Table 1.2: Value and error for each iteration until ε_{min} was met

it.	value	error	order of convergence
1	$[-7.04918e^{-01}, -2.29508e^{-02}]$	$2.00316e^{-02}$	$0.00000e^{01}$
2	$[-7.07106e^{-01}, 2.46445e^{-05}]$	$2.12159e^{-05}$	$2.75180e^{01}$
3	$[-7.07107e^{-01}, -3.01603e^{-14}]$	$1.33890e^{-12}$	$2.54064e^{01}$
4	$[-7.07107e^{-01}, 0.00000e^{01}]$	$1.47902e^{-15}$	$1.34756e^{01}$

Here, the algorithm was used in order to solve for an extremum of $l(u, v)$. The order of convergence implies that the iteration converges quadratically.

Note: Errors have been calculated with the built-in Octave-function `norm()`.

2 Euler-Lagrange equations of two example systems

Different numerical integration methods will be evaluated by investigating the dynamic behaviour of two exemplary dynamical systems. These systems will be modelled as Lagrangian-Systems by deriving an interdependent expression of positions, velocities as well as accelerations within the system by observing their energetical behaviour. The general form of the *Euler-Lagrange* equation, which defines the pivotal relation to this cause, is given as

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{\mathbf{q}}}(\mathbf{q}, \dot{\mathbf{q}}) - \frac{\partial L}{\partial \mathbf{q}}(\mathbf{q}, \dot{\mathbf{q}}) = 0 \quad (2.1)$$

where $L = T - V$ stands for the so-called *Lagrangian*. In this context, T is regarded as the system's kinetic Energy, whereas V describes it's potential Energy. In the following, the planar pendulum with a minimal angle-coordinate $q = \varphi$ as well as the Kepler problem in cartesian space with $\mathbf{q} = (x, y)$ will be investigated. The expresions for their respective kinetic and potential energy are given as follows:

- **Planar Pendulum:**

$$T(\dot{q}) = \frac{1}{2} m l^2 \dot{\varphi}^2, \quad V(q) = -mgl \cos(\varphi) \quad (2.2)$$

- **Kepler Problem:**

$$T(\dot{\mathbf{q}}) = \frac{m}{2} (\dot{x}^2 + \dot{y}^2), \quad V(\mathbf{q}) = -G \frac{Mm}{\sqrt{x^2 + y^2}} \quad (2.3)$$

Obviously, both systems are autonomous as they don't depend on time explicitly. After calculating the *Lagrangian* as well as the *Euler-Lagrange* equation, it makes sense to explicitly solve for $\ddot{\mathbf{q}}$. This enables a subsequent transformation into a partitioned system of first-order ODEs. As the final step, the *Jacobian* of the system was calculated as it is required when using implicit methods in later chapters which were solved using (1.1). Below, the results of applying the described procedure to both systems are listed consecutively.

2.1 Planar pendulum, minimal coordinate: $q = \varphi$

a) **Lagrangian:**

$$L = T - V = \frac{1}{2}ml^2\dot{\varphi}^2 - (-mgl \cos \varphi) = \frac{1}{2}ml^2\dot{\varphi}^2 + mgl \cos \varphi \quad (2.4)$$

b) **Partial Differentiation:**

$$\frac{\partial L}{\partial \dot{\varphi}} = ml^2\dot{\varphi} \qquad \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\varphi}}\right) = ml^2\ddot{\varphi} \qquad \frac{\partial L}{\partial \varphi} = -mgl \sin \varphi$$

c) **Insertion of (2.4) into (2.1):**

$$\begin{aligned} \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\varphi}}\right) - \frac{\partial L}{\partial \varphi} &= ml^2\ddot{\varphi} + mgl \sin(\varphi) = 0 \\ \iff \ddot{\varphi} &= -\frac{g}{l} \sin \varphi \end{aligned}$$

d) **Transformation into system of first order ODEs:**

$$\begin{bmatrix} \dot{\varphi} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \omega \\ -\frac{g}{l} \sin \varphi \end{bmatrix} \quad (2.5)$$

e) **Jacobian of the system:**

$$\Rightarrow J = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} \cos \varphi & 0 \end{bmatrix} \quad (2.6)$$

In the upcoming simulations of this system, the following set of parameters was used:

- Gravitational constant on Earth $g = 9.81 \frac{m}{s^2}$
- Length of the pendulum's rod $l = 1m$
- Mass of the pendulum $m = 1kg$

2.2 Kepler-problem, cartesian coordinates $q = (x, y)$

a) **Lagrangian:**

$$L = T - V = \frac{m}{2}(\dot{x}^2 + \dot{y}^2) - \left(-G \frac{Mm}{\sqrt{x^2 + y^2}}\right) = \frac{m}{2}(\dot{x}^2 + \dot{y}^2) + G \frac{Mm}{\sqrt{x^2 + y^2}} \quad (2.7)$$

b) **Differentiation:**

$$\frac{\partial L}{\partial \dot{q}} = \begin{bmatrix} m\dot{x} \\ m\dot{y} \end{bmatrix} \quad \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}} \right) = \begin{bmatrix} m\ddot{x} \\ m\ddot{y} \end{bmatrix} \quad \frac{\partial L}{\partial q} = \begin{bmatrix} -GMm \frac{x}{\sqrt{x^2 + y^2}^3} \\ -GMm \frac{y}{\sqrt{x^2 + y^2}^3} \end{bmatrix}$$

c) **Insertion of (2.7) into (2.1):**

$$\begin{aligned} \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}} \right) - \frac{\partial L}{\partial q} &= \begin{bmatrix} m\ddot{x} - \left(-GMm \frac{x}{\sqrt{x^2 + y^2}^3}\right) \\ m\ddot{y} - \left(-GMm \frac{y}{\sqrt{x^2 + y^2}^3}\right) \end{bmatrix} = \begin{bmatrix} m\ddot{x} + GMm \frac{x}{\sqrt{x^2 + y^2}^3} \\ m\ddot{y} + GMm \frac{y}{\sqrt{x^2 + y^2}^3} \end{bmatrix} = 0 \\ \Leftrightarrow \begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix} &= \begin{bmatrix} -\frac{GMx}{\sqrt{x^2 + y^2}^3} \\ -\frac{GM y}{\sqrt{x^2 + y^2}^3} \end{bmatrix} \end{aligned}$$

d) **Transformation into system of first order ODEs:**

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{v}_x \\ \dot{v}_y \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ -\frac{GMx}{\sqrt{x^2 + y^2}^3} \\ -\frac{GM y}{\sqrt{x^2 + y^2}^3} \end{bmatrix} \quad (2.8)$$

e) **Jacobian of the system:**

$$\Rightarrow J = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ GM \left(\frac{3x^2}{\sqrt{x^2 + y^2}^5} - \frac{1}{\sqrt{x^2 + y^2}^3} \right) & GM \frac{3xy}{\sqrt{x^2 + y^2}^5} & 0 & 0 \\ GM \frac{3xy}{\sqrt{x^2 + y^2}^5} & GM \left(\frac{3y^2}{\sqrt{x^2 + y^2}^5} - \frac{1}{\sqrt{x^2 + y^2}^3} \right) & 0 & 0 \end{bmatrix} \quad (2.9)$$

In the upcoming simulations of this system, the following values will be used for the parameters:

- Gravitational constant in space $G = 2.959122082860$
- Planetary mass of the sun including the inner planets $M = 1.00000597682$

The values in *SI*-units can be obtained from the *Astronomical Unit* convention.

3 Calculation of the discrete Euler-Lagrange equations for the Kepler problem

The variational integrators take a different approach at solving for kinematic properties of a system. While ODE solvers in general work with a first order system of ODEs by discretizing the right-hand side $\mathbf{f}(t, u)$, the variational integrators discretize the Euler-Lagrange equations of a system by a common numerical integrator and aim to solve numerically for $\mathbf{q}_{k+1}$ by this. Independently from the integrator, the following procedure can be applied to other numerical integration forms.

3.1 Variational integration based on the Midpoint Rule

- Discrete Lagrangian:

$$L_d(\mathbf{q}_k, \mathbf{q}_{k+1}) = L_d((x_k, y_k), (x_{k+1}, y_{k+1})) = \dots \quad (3.1)$$

$$h \left\{ \frac{m}{2} \left[\left(\frac{x_{k+1} - x_k}{h} \right)^2 + \left(\frac{y_{k+1} - y_k}{h} \right)^2 \right] + \frac{GMm}{\sqrt{\left(\frac{x_{k+1} + x_k}{2} \right)^2 + \left(\frac{y_{k+1} + y_k}{2} \right)^2}} \right\}$$

- Discrete Euler-Lagrange equations:

$$0 = D_1 L_d(\mathbf{q}_k, \mathbf{q}_{k+1}) + D_2 L_d(\mathbf{q}_{k-1}, \mathbf{q}_k) \quad (3.2)$$

$$= \begin{bmatrix} \frac{m}{h}(-x_{k+1} + 2x_k - x_{k-1}) - \frac{GMm}{4} \left[\frac{x_{k+1} + x_k}{\sqrt{\left(\frac{x_{k+1} + x_k}{2} \right)^2 + \left(\frac{y_{k+1} + y_k}{2} \right)^2}} + \frac{x_k + x_{k-1}}{\sqrt{\left(\frac{x_k + x_{k-1}}{2} \right)^2 + \left(\frac{y_k + y_{k-1}}{2} \right)^2}} \right] \\ \frac{m}{h}(-y_{k+1} + 2y_k - y_{k-1}) - \frac{GMm}{4} \left[\frac{y_{k+1} + y_k}{\sqrt{\left(\frac{x_{k+1} + x_k}{2} \right)^2 + \left(\frac{y_{k+1} + y_k}{2} \right)^2}} + \frac{y_k + y_{k-1}}{\sqrt{\left(\frac{x_k + x_{k-1}}{2} \right)^2 + \left(\frac{y_k + y_{k-1}}{2} \right)^2}} \right] \end{bmatrix}$$

This iterative set of equations is implicit. The Newton-Raphson algorithm from chapter 1 could be applied when implementing this solver.

3.2 Variational integration based on the Trapezoidal Rule

- Discrete Lagrangian:

$$\begin{aligned} L_d(\mathbf{q}_k, \mathbf{q}_{k+1}) &= L_d((x_k, y_k), (x_{k+1}, y_{k+1})) = \dots \\ &= \frac{m}{2h} \left((x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2 \right) + \frac{hGMm}{2} \left(\frac{1}{\sqrt{x_k^2 + y_k^2}} + \frac{1}{\sqrt{x_{k+1}^2 + y_{k+1}^2}} \right) \end{aligned} \quad (3.3)$$

- Discrete Euler-Lagrange equations:

$$\begin{aligned} 0 &= D_1 L_d(\mathbf{q}_k, \mathbf{q}_{k+1}) + D_2 L_d(\mathbf{q}_{k-1}, \mathbf{q}_k) \\ &= \begin{bmatrix} \frac{m}{h} (2x_k - x_{k+1} - x_{k-1}) - hGMm \left(\frac{x_k}{\sqrt{x_{k+1}^2 + y_{k+1}^2}^3} \right) \\ \frac{m}{h} (2y_k - y_{k+1} - y_{k-1}) - hGMm \left(\frac{y_k}{\sqrt{x_{k+1}^2 + y_{k+1}^2}^3} \right) \end{bmatrix} \end{aligned} \quad (3.4)$$

Obviously this set of equations is explicit, which is more convenient for implementation.

3.3 Post-Processing of results obtained from Variational Integrators

Since the variational integrators only consider the position values, the velocities from (2.8) must be calculated numerically as well by another algorithmic procedure. In the scriptum of this course, the *qq*- and the *qp*-algorithm have been introduced to this case.

For the implementation of the *Variational Integrator* based on the *Trapezoidal Rule*, the *qq*-algorithm was used, as it appeared more intuitive at first glance. A short description of the algorithm can be written as

- 1): Compute q_1 by using the equation $p_0 = -D_1 L_d(q_0, q_1)$
- Use the *Discrete Euler Lagrange* equation, in case of the *Trapezoidal Rule* given by (3.4), to compute $\{(q_k, q_{k+1})\}_{k=1}^{N-1}$

Due to time and complexity reasons, the *Variational Integrator* based on the *Midpoint Rule* was not successfully implemented in time. In the upcoming sections, the implementation of section 3.2 should at least give a general impression of how a variational integrator behaves in comparison to other numerical solvers.

4 Visualization and interpretation of results based on different numerical methods for simulating the example systems

In this chapter, the example systems from chapter 2 were evaluated using different numerical methods for solving a system of ODEs. Each method has a defined accuracy $\mathcal{O}(h)$ and is of a certain order of convergence p , as the exact solution u on a defined space Ω of the problem will only be approximated up to an error $e = |u - u_h|$. Exceptions can occur at some points when the approximate solution u_h has been generated on a grid where the error is 0 on grid points, while in general $\lim_{h \rightarrow \infty} e \rightarrow \infty$ can be observed for stable methods.

This desirable property can be investigated amongst others by varying initial values $x_0 \in \Omega$ of the problems (2.5) and (2.8) or the step-size h for each of the following methods. To this cause, the methods have been implemented in `Octave` as the results from sections 2.1 and 2.2 were passed. Furthermore, short interpretations and assumptions concerning each method were performed based on the obtained results.

4.1 Euler methods

The *Euler* methods are the most basic ones regarding their complexity and difficulty of implementation. While their errors can be bounded depending on the underlying system of ODEs, its convergence can also vary by the starting point and the step size. They should therefore be investigated individually for each system. The order of convergence p_{EE} will be investigated in section 6.

4.1.1 Explicit Euler algorithm

The algorithm has $p = 1$ as will be shown later and can be formulated as

$$x_{k+1} = x_k + h(t_k, x_k), \quad x_0 = x(t_0) \quad (4.1)$$

Obviously, it is a one-step method and only depends on the current value in time.

4.1.2 Implementation and evaluation of the Explicit Euler algorithm

The implementation in `Octave` is straight forward. The initial values for (2.5) and (2.8) as well as the total simulation time have been chosen as follows in a meaningful way to ensure proper analysis of the method:

- Planar pendulum:

$$x_0 = \begin{bmatrix} \varphi_0 \\ \omega_0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad T = 1\text{e}1 = 10 \quad (4.2)$$

- Kepler problem (*Orbiting body: Jupiter*):

$$x_0 = \begin{bmatrix} x_0 \\ y_0 \\ v_{x0} \\ v_{y0} \end{bmatrix} = \begin{bmatrix} -15.5387357 \\ -25.2225594 \\ 0.00276725 \\ -0.00170702 \end{bmatrix}, \quad T = 1\text{e}4 = 10000 \quad (4.3)$$

The following results have been obtained for varying step sizes:

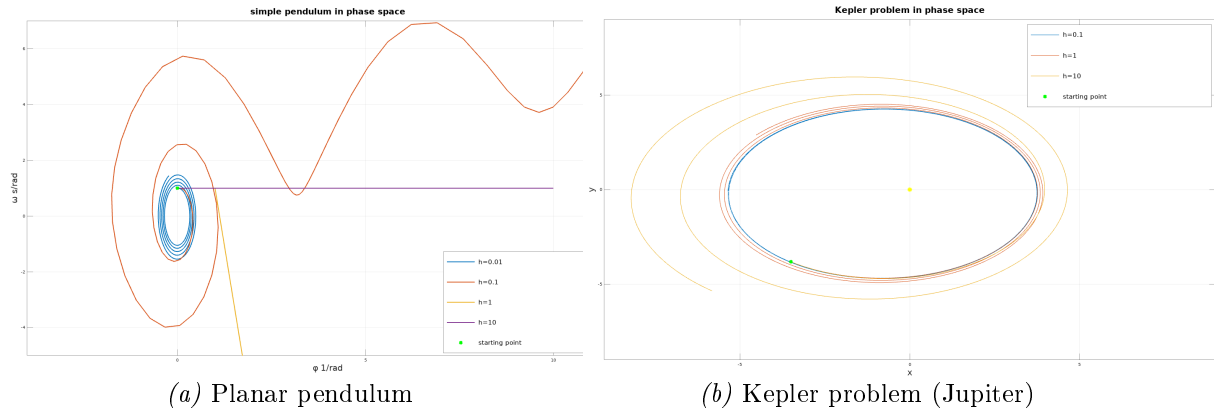


Figure 4.1: Simulation results for different values of h using the Explicit Euler algorithm

While the planar pendulum was primarily investigated in phase space, the xy -position was observed within the Kepler problem. Variations of h show the same tendency for each example system, as the results become less accurate for larger step sizes. For large values, the results may become unusable at all. Furthermore, in all cases it can be observed that the system seems to gain energy, as it deviates away from any expectable trajectory. This raises the question of the overall applicability of the *Explicit Euler* method.

The issue appears to be systematic, as it is reproducible for varying initial values and simulation times. In the next run of the simulation, in case of the planar pendulum the initial values were changed to $x_0 = [1, 0]^T$ and the simulation time to $T = 1\text{e}2 = 100$, while the issue also preserves when looking at the Kepler problem for another orbiting body, longer simulation times as well as smaller step sizes:

- Planar pendulum:

$$x_0 = \begin{bmatrix} \varphi_0 \\ \omega_0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; T = 1\text{e}2 = 100 \quad (4.4)$$

- Kepler problem (*Orbiting body: Neptune*):

$$x_0 = \begin{bmatrix} x_0 \\ y_0 \\ v_{x0} \\ v_{y0} \end{bmatrix} = \begin{bmatrix} 08.3101420 \\ -16.2901086 \\ 0.00354178 \\ 0.00137102 \end{bmatrix}, T = 1\text{e}5 = 100000 \quad (4.5)$$

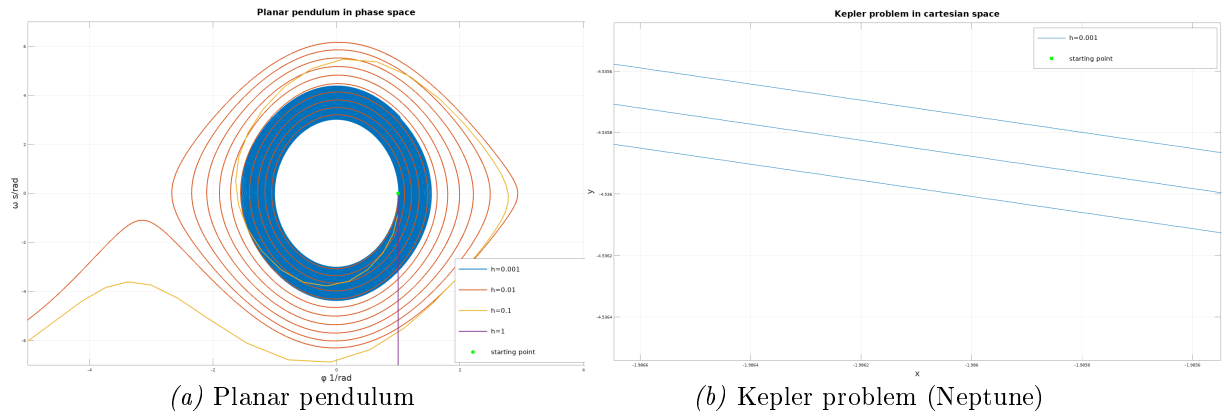


Figure 4.2: Simulation results using Explicit Euler with varying simulation parameters

The phase-space diagram of the alternatively parametrized Kepler problem also suggests that using the Explicit Euler method results in obtaining an unbounded error e with increasing simulation time $T \gg 0$ and step size $h \rightarrow \infty$, even though the issue seems to become less drastic the further away the orbiting body is located from the sun:

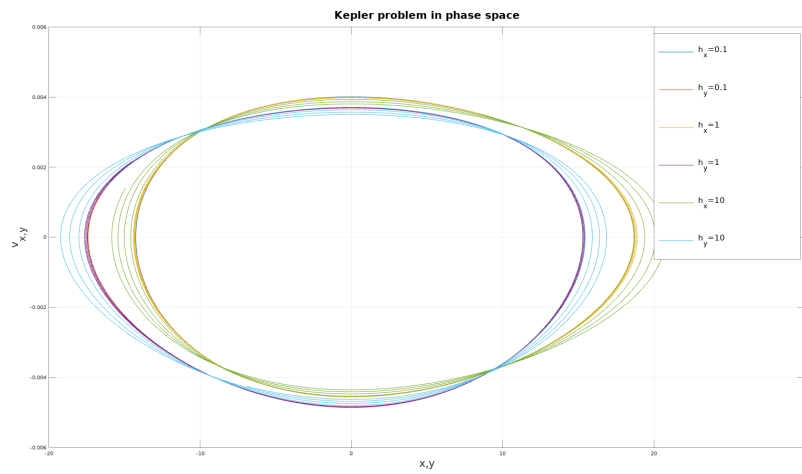


Figure 4.3: Neptune's phase space diagram of the Kepler problem for different step sizes

This may be explained by the fact that outer planets take significantly more time to orbit the sun, which makes them less vulnerable to larger h .

4.1.3 Implicit Euler algorithm

The algorithm is also of convergence order $p = 1$ and can be formulated as shown in

$$x_{k+1} = x_k + h(t_{k+1}, x_{k+1}), \quad x_0 = x(t_0) \quad (4.6)$$

This one-step method refers the unknown value in time of the future step. Therefore, a system of non-linear equations has to be solved. To this cause, the Newton-Raphson algorithm from equation (1.1) can be used after reformulating (4.6) as

$$F(x_{k+1}) = x_{k+1} - x_k - hf(t_{k+1}, x_{k+1}) \quad (4.7)$$

and subsequently taking the derivative of (4.7) whilst setting it to zero:

$$0 = DF(x_{k+1}) = I - Df(t_{k+1}, x_{k+1}) \quad (4.8)$$

The threshold of the stopping criterion used in Newton's method will be set to $\varepsilon = 10^{-12}$, in accordance to chapter 1.

4.1.4 Implementation and evaluation of the Implicit Euler algorithm

As already suggested in 4.1.3, the already implemented Newton-Raphson algorithm can be used to this cause when implementing the method in `Octave`. The Jacobians of the systems required to this cause were derived in (2.6) and in (2.9). In order to compare the implicit method with the explicit one effectively, the same parameter sets as in (4.3) and (4.2) should be and were used for the simulations below:

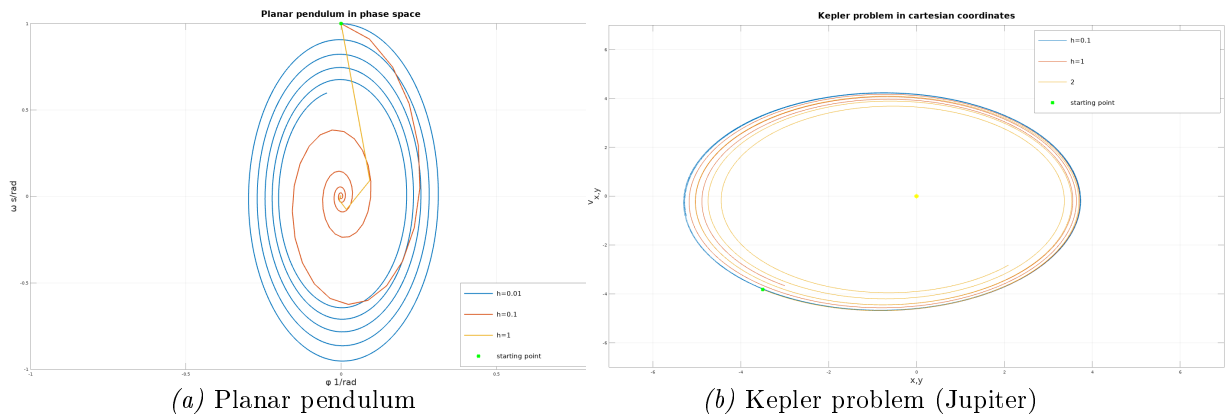


Figure 4.4: Simulation results for different values of h using the Implicit Euler algorithm

Analogously to figure 4.1, the method does not seem to give proper results for the system for any sort of step sizes or parameters. Other simulation runs with different values for h confirm this hypothesis. In contrast to the explicit method, the energy of the system decreases and since $p_{ee} = p_{ie}$, one might assume that the implicit method loses the same amount of energy that the explicit one gains. Even though this will be falsified later on, equal results in regard to $\|e\|$ might be obtained when varying parameters, step sizes and when looking at phase space of the Kepler problem, which can be of use for yielding better numerical solutions.

4.2 Symplectic integrators

The family of symplectic integrators can be used to numerically integrate *Hamiltonian Systems*, which are of the form

$$\begin{aligned}\dot{q} &= +\frac{\partial H}{\partial p}(q, p) \\ \dot{p} &= -\frac{\partial H}{\partial q}(q, p)\end{aligned}\tag{4.9}$$

where q are generalized coordinates, p are the canonical momenta and H is the *Hamiltonian* function. The *Hamiltonian* of a mechanical system represents its total energy and therefore is written in contrast to L in (2.1) as $H = T^H + V$. Using the *Legendre transformation*

$$\begin{aligned}p &= +\frac{\partial L}{\partial \dot{q}} \\ \dot{q} &= \frac{\partial H}{\partial p}\end{aligned}\tag{4.10}$$

one can go from the representation as a *Lagrangian System* to a *Hamiltonian System* and vice versa. Symplectic integrators work with partitioned systems of the form as in (4.9). Hence, one might assume that such integration methods can also be applied to the partitioned systems in (2.5) and (2.8).

4.2.1 Symplectic Euler method

The fact that $p_{EE} = p_{IE}$ can be used to one's advantage in order to obtain more valuable results than in 4.1.2 and 4.1.4. The *Symplectic Euler* method combines (4.1) and (4.8) in each step of the numerical resolution of partitioned systems and thereby mitigates the systematic errors of the explicit and implicit methods. For a general partitioned system, the *Symplectic Euler* method is given as

$$\begin{aligned}u_{k+1} &= u_k + ha(u_{k+1}, v_k) \\ v_{k+1} &= v_k + hb(u_{k+1}, v_k)\end{aligned}\tag{4.11}$$

and obviously uses an *Implicit Euler* step for the first equation and an *Explicit Euler* step for the second one.

4.2.2 Implementation and evaluation of the Symplectic Euler algorithm

If the first equation in (4.11) only depends on the second variable of a system, it can also be solved explicitly. This is the case in both (2.5) and (2.8) and therefore eases the implementation for the given use-cases. Again, (4.2) and (4.3) were used as exemplary parameter sets leading to the following results:

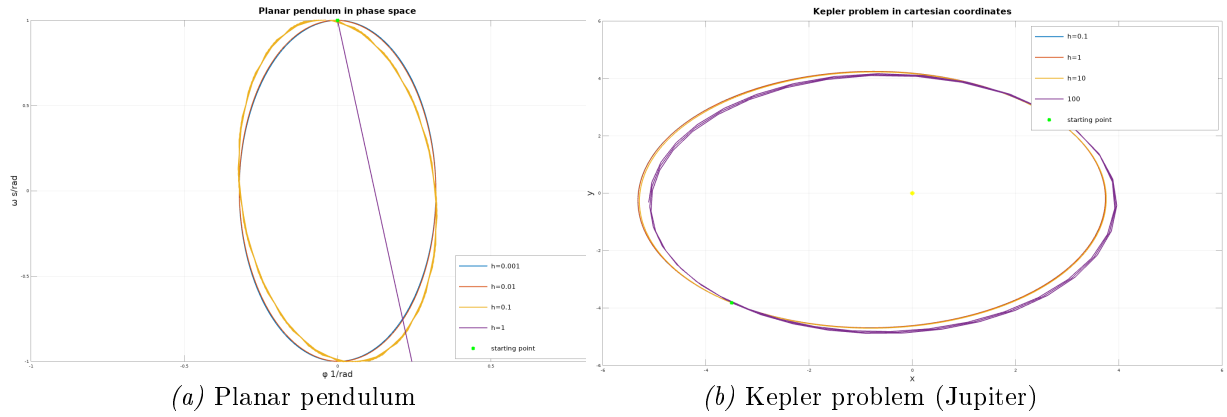


Figure 4.5: Simulation results for different values of h using the Symplectic Euler algorithm

Applying the *Symplectic Euler* algorithm leads to reasonable results for both systems on an abstract level, especially for smaller h . For larger step sizes, interchanging sections with energy gainings and losses can be observed:

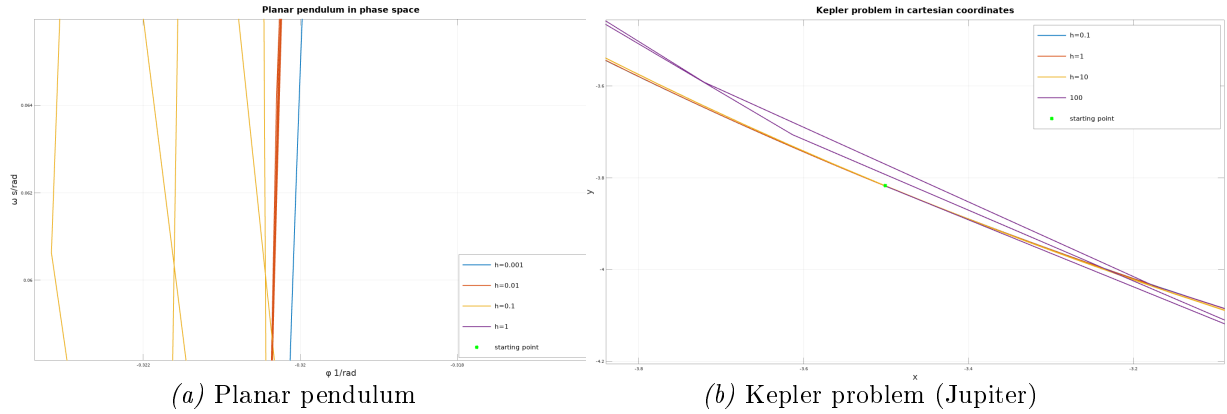


Figure 4.6: Closeups of the results for using the Symplectic Euler algorithm

4.2.3 Störmer-Verlet method

Another symplectic integrator is the *Stoermer-Verlet* algorithm. Unlike the other methods investigated so far, it is a two-step method, meaning that two steps q_{k-1} , q_k are required to

compute q_{k+1} . The algorithm is given as

$$\begin{aligned} q_{k+1} &= 2q_k - q_{k-1} + h^2 f(q_k) \\ q_1 &= q_0 + hv_0 + \frac{h^2}{2} f(q_0) \end{aligned} \quad (4.12)$$

Since the velocity is not calculated by this method, it has to be approximated independently, e.g. by a midpoint rule. Furthermore, the first step has to be calculated externally, e.g. with an *Explicit Euler* step. Alternatively, the *Stoermer-Verlet* algorithm can therefore also be reformulated as a one-step method, which requires more computations and is given as

$$\begin{aligned} v_{k+\frac{1}{2}} &:= v_k + \frac{h}{2} f(q_k) \\ q_{k+1} &= q_k + hv_{k+\frac{1}{2}} \\ v_{k+1} &= v_{k+\frac{1}{2}} + \frac{h}{2} f(q_{k+1}) \end{aligned} \quad (4.13)$$

In theory, both the one-step and the two-step methods yield equal results.

4.2.4 Implementation and evaluation of the Störmer-Verlet algorithm as a one-step method

In order to avoid dependence on the *Explicit Euler* algorithm, the one-step version of the *Stoermer-Verlet* algorithm has been implemented. Once again using (4.2) and 4.3) as parameter sets, the following results were obtained:

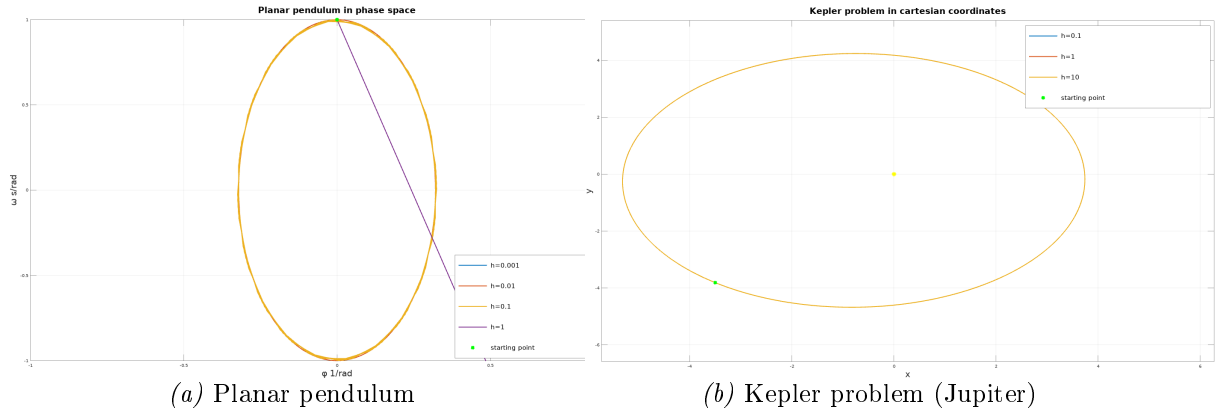


Figure 4.7: Simulation results for different values of h using the Störmer-Verlet algorithm

The algorithm seems to yield acceptable numerical results, at least from a certain threshold step size onwards. In close-ups, a stepping behaviour of the method can be observed in a similar way as in figure 4.6, even though the deviations become apparent only on a smaller macroscopic scale: While the deviation for the planar pendulum for $h = 0.1$ seems to be about $\|e\| \approx 0.05$, it appears to be only half as large for the *Stoermer-Verlet* method. The

significance of this deviation must be evaluated for each application of interest individually and can be reduced drastically for smaller h , while cause and validity of this hypothesis remain to be proved.

Hence, as a part of this investigation it was of interest to compare the behaviour as well as the results of both symplectic methods that have been implemented. A visual comparison is given by the following figure:

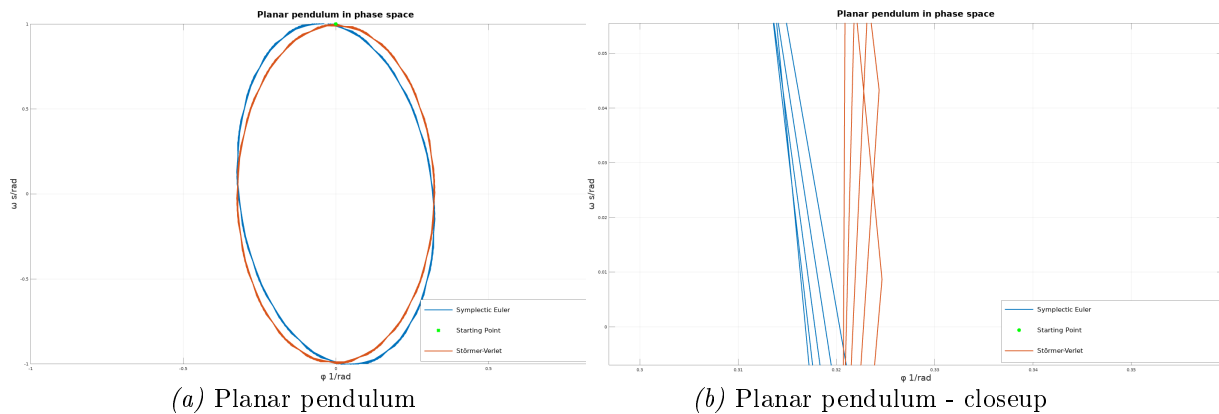


Figure 4.8: Simulation results for the planar pendulum in phase space with $h = 0.1$ using both the Störmer-Verlet and the Symplectic-Euler methods

The first thing that becomes obvious when comparing the results is that the deviation $\|e\|$ only seems to differ periodically in the mentioned way and also in both directions, while the average deviation for both methods seems to be equal up to a certain degree. It is more likely that such deviations in general result from floating-point operations on the computer used to run the `Octave` programs.

Secondly, the phase space trajectories generated by the methods differ on a majority of their curves. From this observation, the question arises which method yields results closer to the analytical solution and by how much the individual methods differ from the latter.

4.3 Runge-Kutta methods

After having a look at methods which are able to yield proper physical results for partitionable systems, the question arises whether such properties can also be realized within one-step methods for non-partitionable systems as such methods would have computational advantages. To address this matter, the course introduced the family of *Runge-Kutta* methods.

4.3.1 The general Runge-Kutta method

Each Runge-Kutta method consists of s internal steps, while only one step is computed externally. An s -stage Runge-Kutta method is given in general form as

$$\begin{aligned} k_i &= f\left(t_0 + c_i h, x_0 + h \sum_{j=1}^s a_{ij} k_j\right) \text{ for } i = 1, \dots, s \\ x_1 &= x_0 + h \sum_{i=1}^s b_i k_i \end{aligned} \quad (4.14)$$

where b_i, c_i are real numbers. $a_{ij} \in \mathbb{R}$ are the entries of a matrix $A \in \mathbb{R}^{s \times s}$. With A, b, c , one can define any Runge-Kutta method by the so-called *Butcher Tableau*:

$$\begin{array}{c|ccc} c_1 & a_{11} & \dots & a_{1s} \\ \vdots & \vdots & \ddots & \vdots \\ c_s & a_{s1} & \dots & a_{ss} \\ \hline & b_1 & \dots & b_s \end{array} \iff \begin{array}{c|c} c & A \\ \hline & b^T \end{array}$$

If A is strictly lower-triangular, the *Runge-Kutta* method is explicit and each k_i can be solved for iteratively. Otherwise, the method is implicit and a nonlinear system of equations must be solved in order to obtain each k_i . Several numerical methods, including the ones investigated in chapter 4.1 can be written by the scheme in (4.14), while the ones in chapter 4.2 belong to the family of partitioned *Runge-Kutta* methods.

4.3.2 Implementation of the general Runge-Kutta method

The implementation of the general *Runge-Kutta* method in `Octave` was split into two parts: While an extra routine was written for explicit methods in order to save computational resources, the *Newton-Raphson* algorithm was once again applied to solve for each k_i in case the passed Butcher-matrix contains a non-strictly-diagonal A . The algorithm is applied in each stage s iteratively until a precision of $\varepsilon = 10^{-12}$ is met, resulting in a larger computational effort. Special attention in order to write a generally applicable function was paid to the fact that k is multidimensional for the Kepler problem, as can be seen in (4.14) when applying it to the Kepler problem. For applying 1.1, the Jacobian has to be computed. It is of the form

$$J_{RK} = \begin{pmatrix} j_{11}DF & \dots & j_{1s}DF \\ \dots & \ddots & \dots \\ j_{s1}DF & \dots & j_{ss}DF \end{pmatrix},$$

where in our case DF is either (2.6) or (2.9) and j_{ij} is a constant scalar. Hence, J_{RK} consists of $s \times s$ blocks of the specifically evaluated model Jacobians times a constant factor.

4.3.3 Evaluation of the general Runge-Kutta method by the RK4 method

The *Classical Runge-Kutta* method is also known as the *RK4* method, as it consists of four internal stages and thus has fourth order accuracy $O(h^4)$ while it is explicit. The method is defined by the *Butcher Tableau* of the form

0				
$\frac{1}{2}$	$\frac{1}{2}$			
$\frac{1}{2}$	0	$\frac{1}{2}$		
1	0	0	1	
	$\frac{1}{6}$	$\frac{2}{6}$	$\frac{2}{6}$	$\frac{1}{6}$

which will be passed to the implemented method from chapter 4.3.3. The parameter sets from (4.2) and (4.3) were also taken as input. The results for both systems are displayed in the following figure:

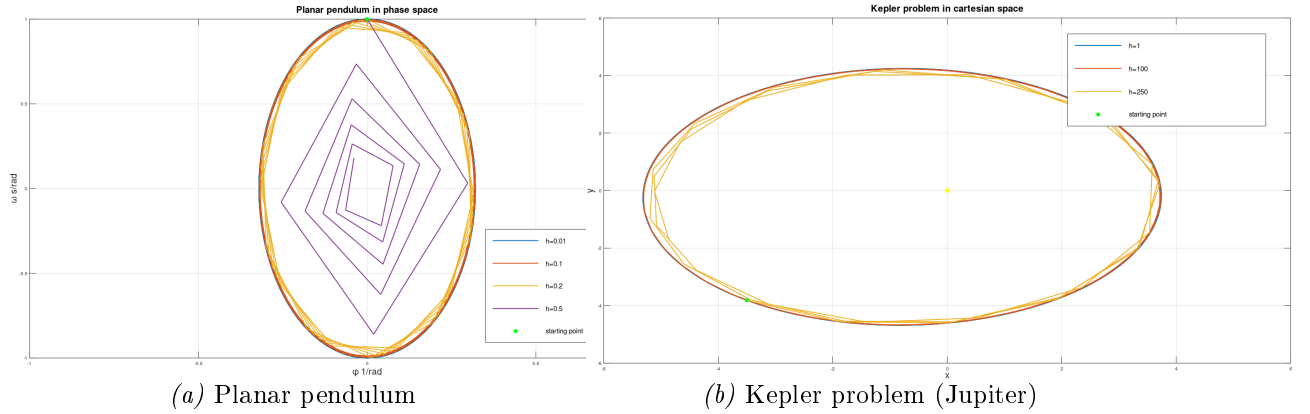


Figure 4.9: Simulation results for different values of h using the RK4 algorithm

For both systems, the RK4 method only yields physically plausible solutions after using a certain threshold value for h , while the results get better for smaller h .

4.3.4 Evaluation of the general Runge-Kutta method by the Radau-IIA method

The *RadauIIA* method is an implicit three-stage collocation method and given by the following *Butcher Tableau*:

$(4 - \sqrt{6})/10$	$(88 - 7\sqrt{6})/360$	$(296 - 169\sqrt{6})/1800$	$(-2 + 3\sqrt{6})/225$
$(4 - \sqrt{6})/10$	$(296 - 169\sqrt{6})/1800$	$(88 - 7\sqrt{6})/360$	$(-2 + 3\sqrt{6})/225$
1	$(16 - \sqrt{6})/36$	$(16 - \sqrt{6})/36$	$1/9$
	$(16 - \sqrt{6})/36$	$(16 - \sqrt{6})/36$	$1/9$

which will be passed to the implemented method from section 4.3.3. The parameter sets from (4.2) and (4.3) were also taken as input. The results for both systems are displayed in the following figure:

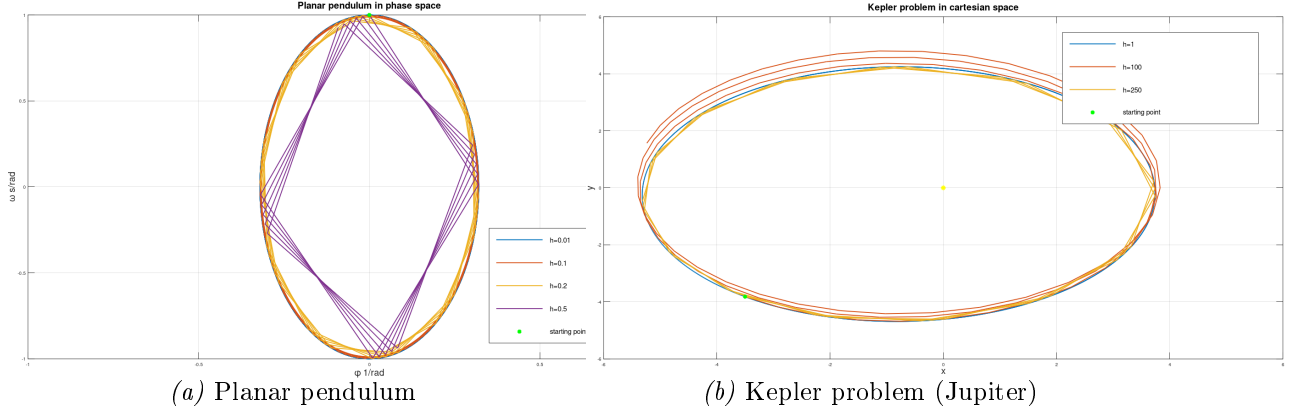


Figure 4.10: Simulation results for different values of h using the Radau-IIA algorithm

While the results for larger h deviate significantly from plausible curves, the result become slightly better than the ones in figure 4.9. Both phenomena can be explained by the fact that collocation methods aim to interpolate the points of the numerical solution u_h . A better impression of the different natures of both the $RK4$ - and the $RadauIIA$ methods might be obtained by visualizing multiple u_{RK4} as well as $u_{RadauIIA}$ in the same plot:

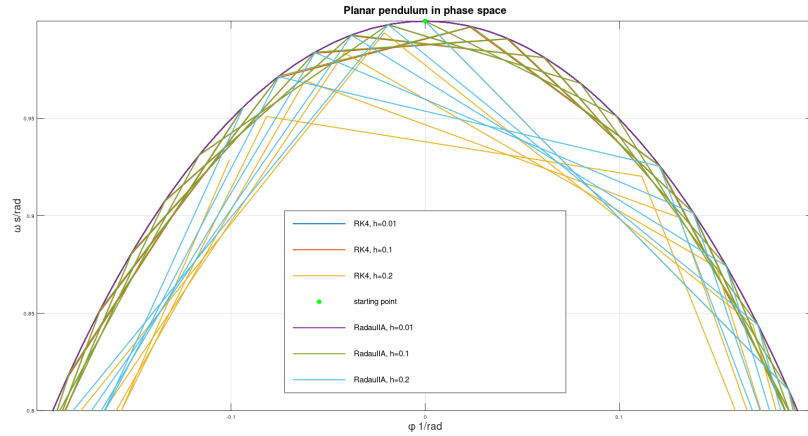


Figure 4.11: Visualization of results obtained from applying the pendulum system to the $RK4$ -method as well as the Radau-IIA method for exemplary values of h

While the difference was quite obvious for larger h , it becomes less apparent for smaller h , until the solution appears to be satisfiable for using both methods. The advantages of each method have to be analyzed separately for specific applications.

4.4 Variational integrators evaluated by the Kepler problem

Since the *Variational Integrator* based on the *Trapezoidal Rule* for the Kepler problem has an explicit scheme for the *Discrete Lagrangian*, the implementation in *Octave* should be intuitive. The implementation is comparable to the implementation of a regular two-step method, as a separate solver has to be used for the first step. In this case, the *Explicit Euler* algorithm has been applied for the first step.

The implementation of the necessary *qq*-algorithm was conducted in accordance to (3.3), (3.4) and section 3.3. Using the set of parameters in (4.3) the following results have been obtained:

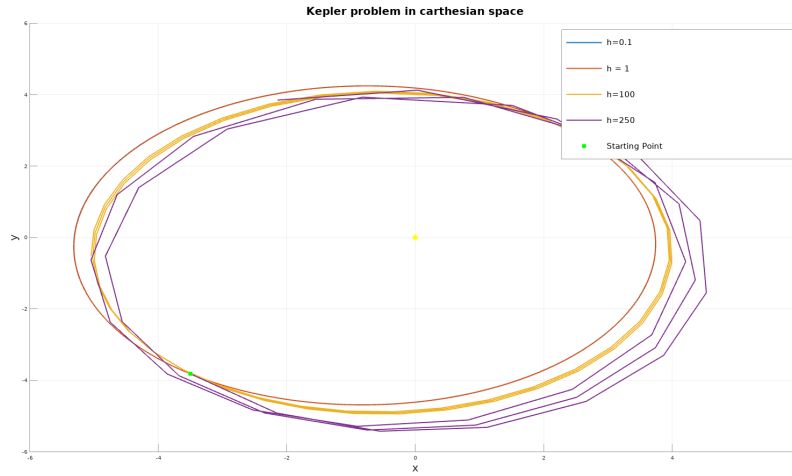


Figure 4.12: Simulation results for different values of h using the Variational Integrator based on the Trapezoidal Rule (Jupiter)

While the trajectory deviates heavily from where to analytical one is suspected to be at, the method returns surprisingly well trajectories starting from $h \approx 10$. This might be due to the limited order of convergence of the underlying *Trapezoidal Rule*. The results for the *Variational Integrator* based on the *Midpoint Rule* might even be worse for larger h , since it's approximation is less accurate by common knowledge, but this is sadly just an assumption since the implementation was not conducted successfully. Better results might be obtained using methods with higher order of convergence, such as the *Simpson Rule*.

4.5 Comparison of numerical solutions with the analytical

The analytical solution for the Kepler problem in (2.8) will be calculated in polar coordinates, where a generalized expression for the radius r of the orbiting body with mass m to the sun with mass M is given as

$$r(\varphi) = \frac{p}{1 + \varepsilon \cos(\varphi - \hat{\varphi})} \quad (4.15)$$

where $\hat{\varphi}$ is an integration constant and

$$p = \frac{p_\varphi^2}{m\alpha}, \quad \text{with} \quad p_\varphi = \frac{\partial L}{\partial \dot{\varphi}}, \quad \alpha = GMm$$

$$\varepsilon = \sqrt{1 + \frac{2Hp_\varphi^2}{m\alpha^2}}, \quad \text{with} \quad H = T + V$$

with T, V, G, M taken from chapter 2.2. The comparison of this analytical solution with different numerical solutions will be conducted by looking at Neptune as the orbiting body with corresponding initial values from (4.5) and $m = 4.37273164546 * 10^{-5}$.

Plotting the results for the analytical solution and the numerical ones generated by the *Explicit*, *Implicit* and *Symplectic Euler* algorithms renders as

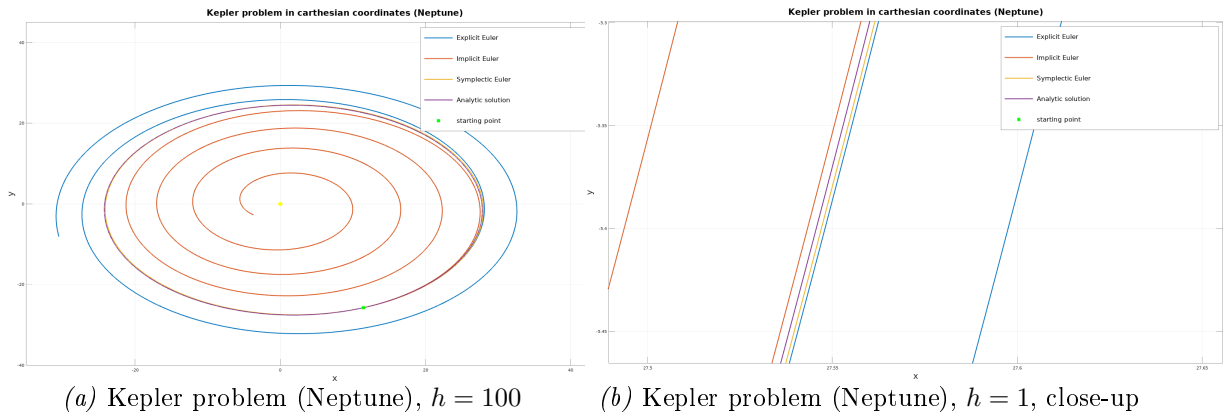


Figure 4.13: Simulation results after computing the numerical solutions of the Euler methods in contrast to the analytical solution

Only the symplectic method follows the trajectorial tendency of the analytical solution for all h , which could be guessed beforehand after looking at chapters 4.1 and 4.2. As the other methods yielded similar solutions for smaller h , it can be assumed that they match the analytical solution equally well for small h .

4.6 Summary

After analyzing various integrators, a step size interval of $h = [1, \dots, 10]$ seems to be the upper bound to yield physically plausible trajectories for the Kepler problem in general, testing different planets and initial conditions. Neglecting the *Explicit/Implicit Euler* methods, choosing $h < 1$ almost certainly yields good results at first glance, which leads to the question how the *Symplectic* methods, *Runge-Kutta* methods and *Variational integrators* can further be distinguished regarding their advantages and disadvantages.

The planar pendulum is better suited to show behaviour for different initial values, but mostly the behaviour is plausible starting from $h = 0.1$ for all methods, again not considering *Explicit/Implicit Euler* methods.

5 Conserved quantities of the Kepler problem

The conservation of system quantities is an essential property of numerical methods for solving ODEs in the sense of their general applicability. In dynamical systems such as in sections 2.1 and 2.2, the constants of motion are examples for quantities to be conserved. For all following simulations, the parameter set for Jupiter was used (see 4.3) while T_{sim} and h were adapted specifically for each method, minding investigational as well as performance reasons.

5.1 Conservation of energy

Looking at the Kepler problem, the kinetic energy T and potential energy V calculate as given in (2.4), while the total energy computes as $H = T + V$.

In the following, the energetical error

$$e_E = |H_u - H_{u_h}| \quad (5.1)$$

will be computed, where H_u stands for the analytically computed, time-invariant total energy of the system and H_{u_h} for the energy computed in each step of the applied numerical algorithm. Plotting the error over the total simulation time results in different errors for different methods:

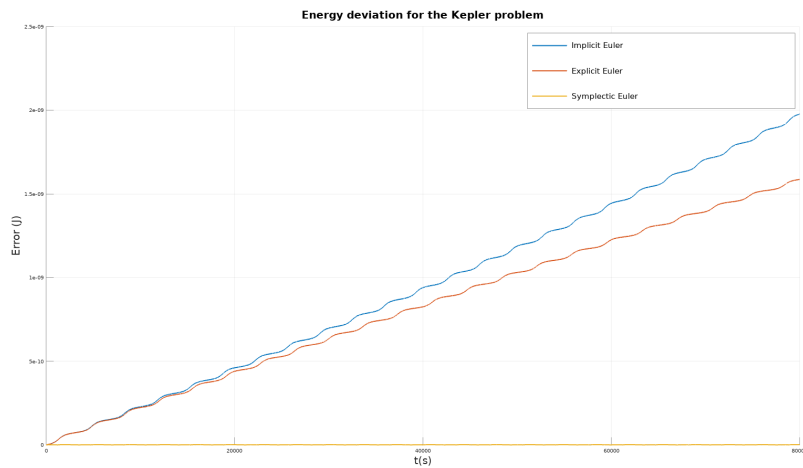


Figure 5.1: Evolution of the energetical error for the Kepler problem calculated by using Euler methods, $h = 0.1$ (Jupiter). ERROR: y-label should be e_E (*)

As expected, the error steadily increases for the *Explicit Euler* method as well as the *Implicit Euler* method, while it seems to be non-existent for the *Symplectic Euler* method. Looking at the family of symplectic methods falsifies any jumping to conclusions:

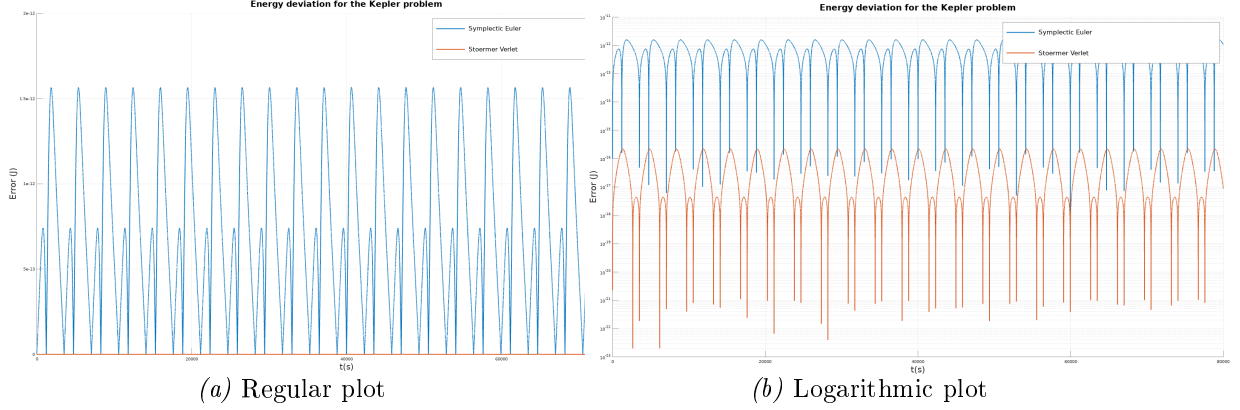


Figure 5.2: Evolution of the energetical error for the Kepler problem calculated by using symplectic methods, $h = 0.1, T = 8e4$ (Jupiter). ERROR (*)

Looking at the logarithmic plot, one can see that the energy behaves differently for the symplectic methods, both in scale as well as in shape. Of greater interest is the fact that the *Störmer Verlet* method yields a maximum energetical error of only $e_E \approx 10^{-16}$. In order to properly validate the scale for varying h , the error behaviours of the Runge-Kutta methods as well as the variational integrators need to be set in comparison:

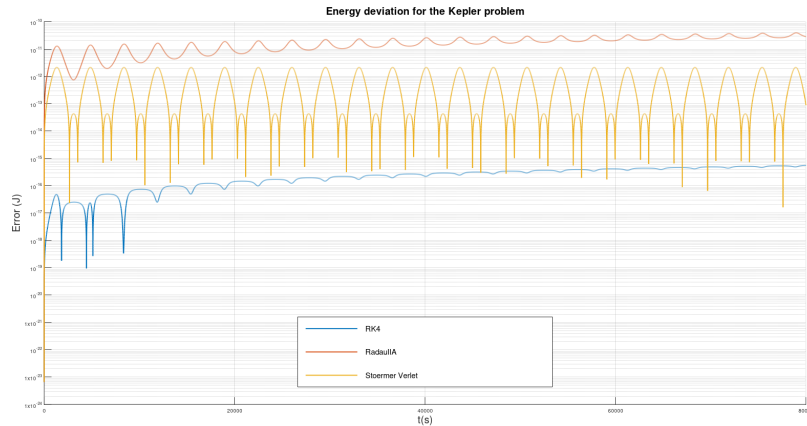


Figure 5.3: Energy error evolution for the Kepler problem calculated by Runge Kutta methods and the Störmer Verlet method, $h = 10, T = 8e4$, Jupiter. ERROR (*)

While the classical *Runge-Kutta* method appears to have the lowest average error, it follows a trend just like the *RadauIIA* method does by seemingly increasing the error, but on a relatively small scale. Nevertheless, this might be an issue for long simulation times, which can certainly be the case for this problem. A convenient observation is that the maxima

and minima of the errors for both *Runge-Kutta* methods do not deviate as drastically, as the oscillations have a smaller amplitude.

5.2 Conservation of linear and angular momentum

Another constant of motion in dynamical systems is the angular momentum L . Since the general equation for a *Hamilton System* such as in (4.11) is given as

$$L = \sum_{i=1}^N q_i \times p_i \quad (5.2)$$

where p equals the linear impulse. It can be obtained applying the *Legendre transformation* in (4.12), which results in

$$p = \frac{\partial L}{\partial \dot{q}} = m \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} p_x \\ p_y \end{bmatrix} \Rightarrow p_{res} = \sqrt{p_x^2 + p_y^2} \quad (5.3)$$

inserting (2.7). Inserting this expression in (5.2) results in

$$L = q \times p = \begin{bmatrix} x \\ y \end{bmatrix} \times m \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = m(x\dot{y} - y\dot{x}) = xp_x - yp_y \quad (5.4)$$

as the system is two-dimensional in the xy -plane.

Plotting the error e_p in analogy to figure 5.5a over the total simulation time results in different errors for different methods:

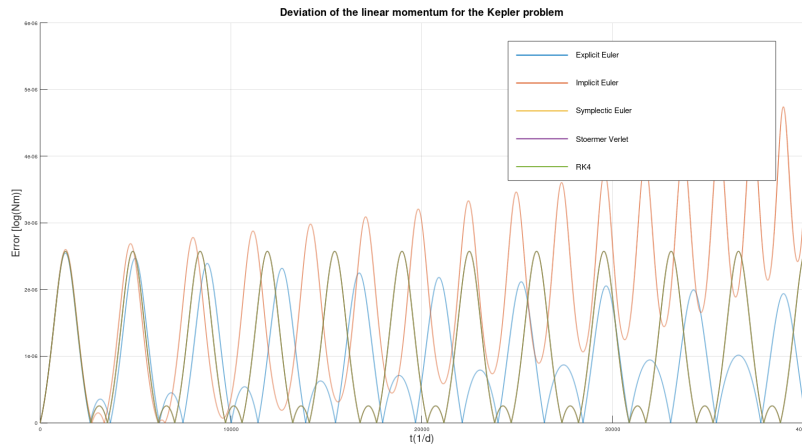


Figure 5.4: Error evolution for the linear momentum for the Kepler problem calculated by Runge Kutta methods, $h = 1$, $T = 4e4$, Jupiter

All methods yield a similar error of $e_p \approx 1e - 6$ and oscillate significantly. Plotting the same results in a y-logarithmic plot does not provide valid information as the oscillations

deviate between $1e-5 > e_p > 1e-22$. This can in part be explained by limitations with regard to machine precision: Working on a conventional 64-bit machine software using IEEE floating point numbers of internal type `signed float64` as `Octave` does results in a precision for arithmetic operations of the kind shown in (5.1) of 10^{-16} . Obtaining $e < 10^{-16}$ will therefore cause Bit shifting, which may result in `sign` errors for the first bit of `float64` variables. This is especially of concern when working with logarithmic plots and has to be kept in mind for the following investigation of preservation of angular momentum. Parts of the behaviours in figure 5.3 can also be explained by this.

Plotting the error e_L in analogy to figure 5.2 over the total simulation time results in different errors for different methods:

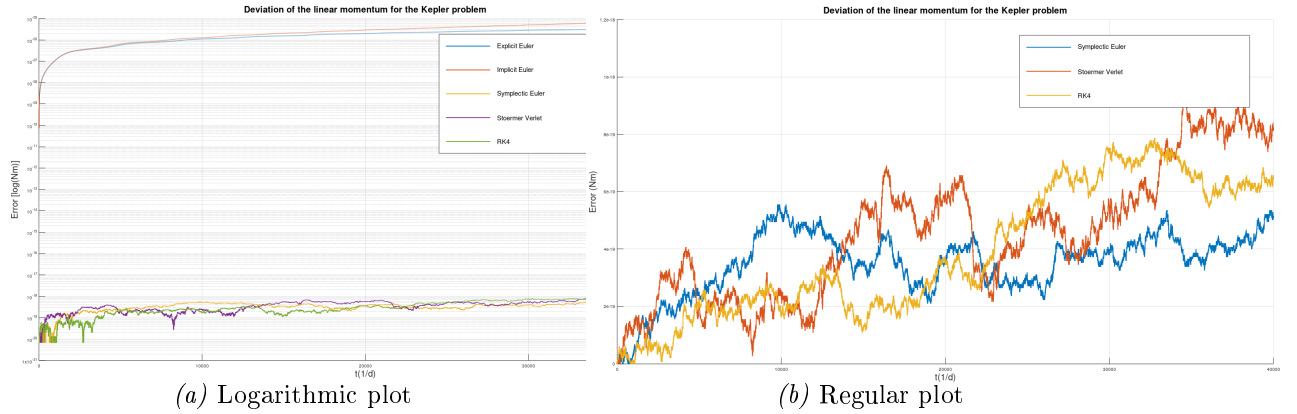


Figure 5.5: Error evolution for the angular momentum of the Kepler problem calculated by using different methods, $h = 1, T = 4e4$ (Jupiter)

While the *Explicit* and *Implicit Euler* methods yield significant errors, the symplectic as well as the *Runge-Kutta* methods appear to preserve the angular momentum in the sense of machine precision. Oscillations and trends are on a small scale and can also be explained by the latter.

5.3 Conservation of physical quantities by the VI-TP integrator in a general context

To deepen the analysis on property-preservation further, the evolution of physical quantities during application of the *VI-TP* method from chapter 3 will be analyzed. The *RadauIIA* method, of which the energetical behaviour is already known, will serve as contrast. In order shift the analytical approach from the last sections, all physical quantities will be shown simultaneously in the following plots for the already known parameters $h = 1, T = 4e4$ and $h = 10, T = 8e4$ (see figures 5.3 and 5.6):

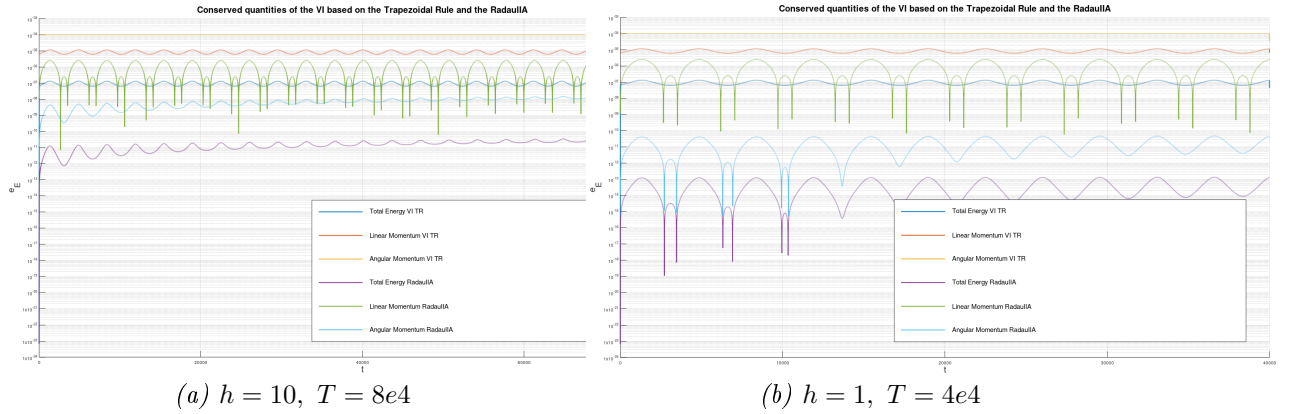


Figure 5.6: Evolution of physical quantities for numerical computation using the VI-TR and RadauIIA methods

Both methods do not preserve the linear momentum. While the *RadauIIA* method conserves energy quite well, it does not conserve the angular momentum as well as the *Classical Runge-Kutta* method. The *VR-TR* method has intermediate energy conservation, while the error for the angular momentum is quite large in comparison to the other methods.

5.4 Summary

From figure 5.6 it can be seen that a variation in step size and simulation time do not impact conservation. In fact, the simulation time should be chosen long enough in order to detect trends, like for the *Runge-Kutta* methods. The fact that linear momentum is not preserved by any method matches theoretical results for model systems obtained during the course. Regarding total energy and angular momentum, some vague assumptions can be made after this rather short investigation. Nevertheless, the following conclusion can be maintained quite confidently:

Table 5.1: Qualitative conclusion on conservation of properties

Integrator	Conserves E	Conserves L
Reg. Euler	No	No
Sympl. Euler	Rather Bad	Very Good
S-V	Very Good	Very Good
RK4	Good (Trend)	Very Good
RadauIIA	Good (Trend)	Good
VI-TR	OK	No

A proper quantitative evaluation should be done in a deeper investigation.

6 Numerical analysis on symmetry and order of convergence

To round up this report up, the symmetry as well as the order of convergence of all considered methods will be investigated. While symmetry is not a necessary property for a numerical solver to yield good results, it can be useful for analyzing a system thoroughly. In contrast, the order of convergence of a method directly correlates with the quality of it's solution in many cases.

6.1 Symmetry of numerical ODE solvers

A one-step method Φ_h is said to be symmetric or time-reversible, if

$$\Phi_h \circ \Phi_{-h} = Id \iff \Phi_h = \Phi_{-h}^{-1} \quad (6.1)$$

While an alytical investigation can give the exact answer whether a numerical ODE solver is symmetric, the investigation will be conducted numerically by reverting h to $-h$ and T to $-T$ within the implementation of each method from chapter 4. While issues regarding machine precision can occur, the general tendency regarding the symmetry of each method should be obtainable in relative and absolute manner.

It should be mentioned that a graphical solution is not sufficient, as errors with significant impact can still occur on a smaller scale. This again depends on the application. To adress this matter, a quantitative error analysis will be conducted for each method.

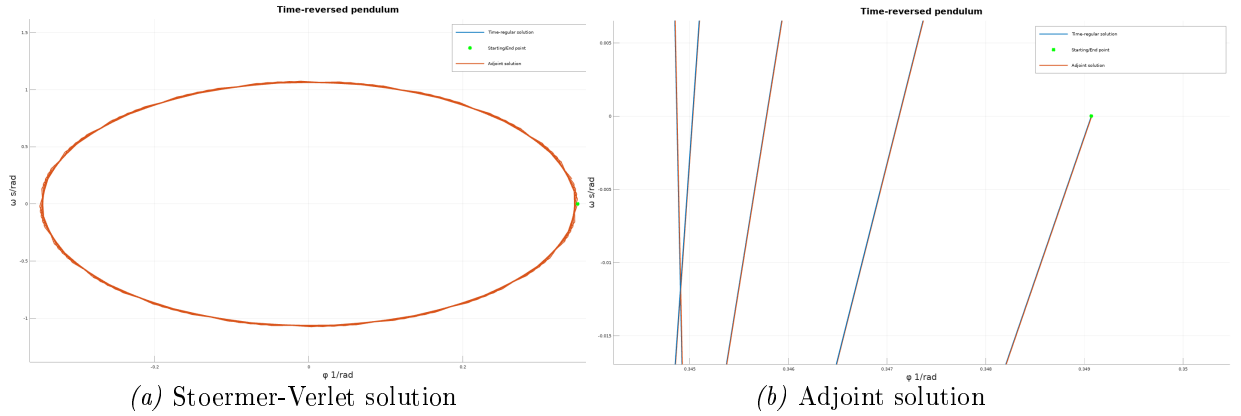
The procedure will exemplarily be executed for the *Stoermer-Verlet*, *Implicit Euler* and the *Classical Runge-Kutta* methods, while the parameter set in use for various h is:

$$x_0 = \begin{bmatrix} \varphi_0 \\ \omega_0 \end{bmatrix} = \begin{bmatrix} \frac{\pi}{9} \\ 0 \end{bmatrix} \quad (6.2)$$

$$T = 1\text{e}1 = 10$$

6.1.1 Symmetry of the Stoermer-Verlet method

Applying the described procedure to the *Stoermer-Verlet* algorithm results in the following solutions for regular and reversed time progression:

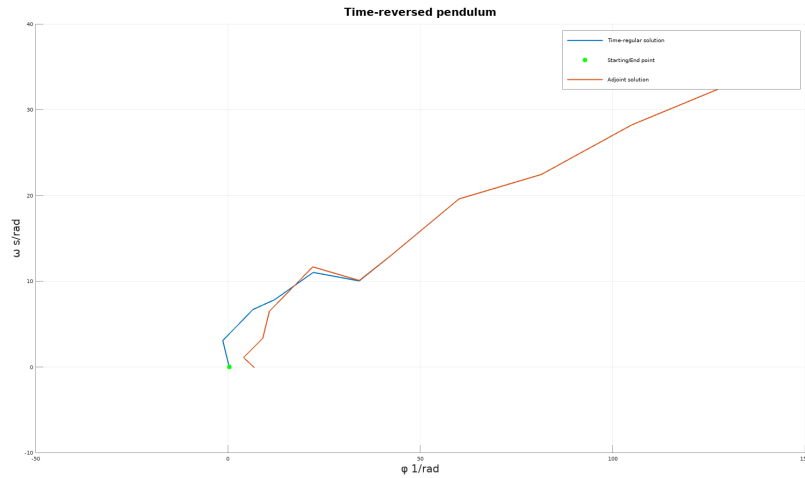
Figure 6.1: Subsequent application of the Störmer-Verlet algorithm and its adjoint, $h = 0.1$

The initial point $x_{0,sv}$ does indeed seem to be matched by the end point of the numerical adjoint $x_{N,adj}$. Nevertheless, the error analysis will be conducted for various h : What is interesting

Table 6.1: Numerical adjoint errors for the Stoermer-Verlet method for different h

h	error φ	error ω
0.001	0	$7.3164e - 08$
0.01	0	$8.0055e - 06$
0.1	0	$1.1644e - 05$
1	0	$6.582e + 00$

to see is that for the first three experiments in the table the error always arises in the velocity component of the adjoint solution. Only for $h = 1$ the position also starts to deviate. This may be explained by the observation that the behaviour is not yielded correctly and that the solutions start to deviate at some point:

Figure 6.2: Subsequent application of the Störmer-Verlet forwards and backwards, $h = 1$

Even though small errors can be observed, one can conclude that the *Stoermer-Verlet* method does indeed show symmetric behaviour, which would be suiting theoretical results, where it can be shown analytically that this method is indeed symmetric. The same holds for research on *LobattoIIIA-IIIB* pairs, which are symmetric Runge-Kutta methods based on the *Stoermer-Verlet* method.

6.1.2 Symmetry of the implicit Euler algorithm

The same procedure will be applied to the *Implicit Euler* method. In theory, the adjoint of the algorithm can be derived as

$$\begin{aligned}\Phi_{-h}^{-1} &:= x_k = x_{k+1} - h(x_k) \\ \iff x_{k+1} &= x_k + hx_k\end{aligned}\tag{6.3}$$

which equals the algorithm for the explicit Euler algorithm. Hence, energy being lost by the implicit Euler method should be joined again by the adjoint. Subsequent application of the implicit and then explicit method with the latter starting starting from $x_{EE,N+1}$ should result in the curve shown in 6.3a:

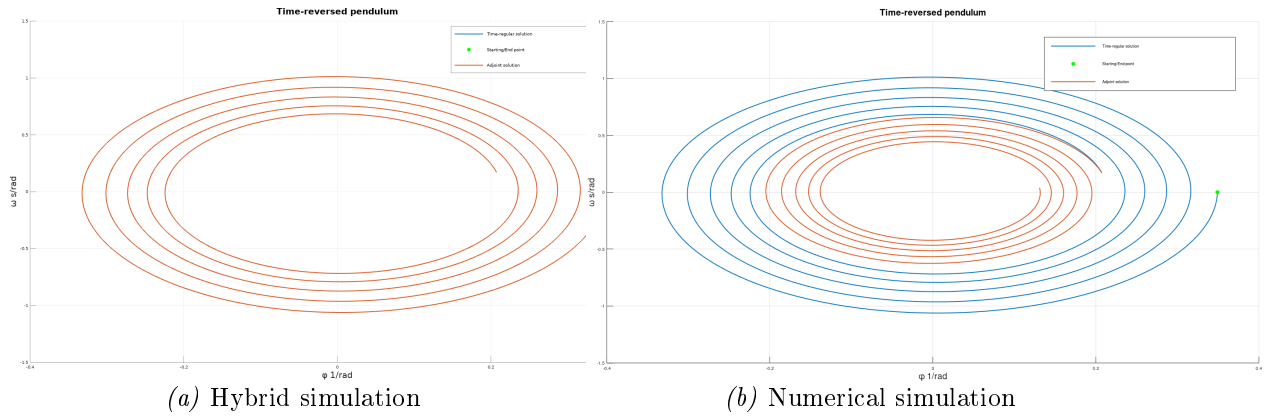


Figure 6.3: Subsequent application of the implicit Euler algorithm and its analytical and numerical adjoint, $h = 0.01$

The numerical analysis returns the following significant errors though as can already be guessed by looking at figure 6.3b:

Table 6.2: Numerical adjoint errors for the Implicit Euler method for different h

h	error
0.001	$3.8465e - 02$
0.01	$2.2167e - 01$
0.1	$3.4904e - 01$
1	$3.4904e - 01$

Errors have been calculated using the function `norm()`. The values clearly shows that symetry is not given for the *Implicit Euler* algorithm at all, with (6.3) being false. Interestingly one can see, that the error norm converges to $3.4904e - 01$ for $h \rightarrow \infty$, which can be explained by the fact that $x_{IE,N+1}$ converges to 0 in this case due to the total loss of energy.

6.1.3 Symmetry of the classical Runge-Kutta method

Nevertheless, the classical Runge-Kuta method will be analyzed numerically applying (6.1). The results are initially shown graphically:

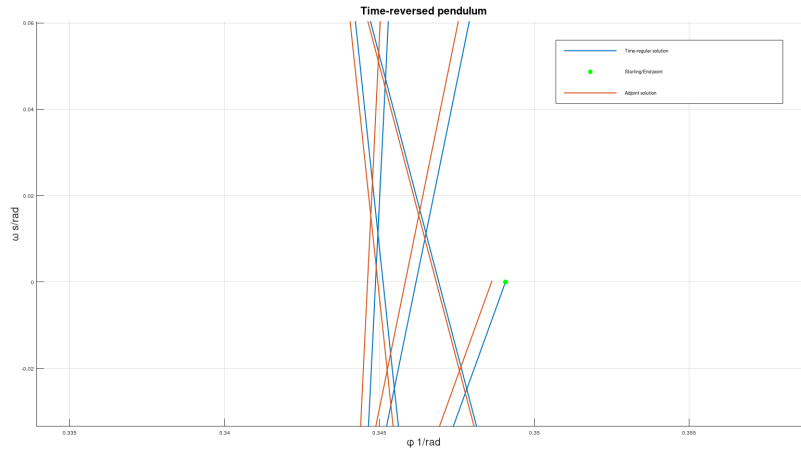


Figure 6.4: Subsequent application of the classical Runge-Kutta algorithm and its numerical adjoint, $h = 0.1$

While a larger deviation can already be observed on a relatively small scale, the errors result as follows:

Table 6.3: Numerical adjoint errors for the Classical Runge-Kutta method for different h

h	error φ	error ω
0.001	$1.0000e - 16$	$3.6589e - 14$
0.01	$4.4094e - 07$	$3.2553e - 09$
0.1	$4.3521e - 04$	$3.2053e - 04$
1	$3.1642e - 01$	$1.70212e + 00$

In comparison to 6.2, the errors are small and also present for φ . Interestingly, the numerically calculated curves deviate for every point in phase-space for $h \geq 1$, unlike in figure 6.1b. Changing x_0 does not change the outcome significantly.

Therefore, one can assume that the *Classical Runge-Kutta* is not symmetric, at least when compared to the *Stoermer-Verlet* method. This statement can be backed up by theoretical

investigations in the scriptum of this course, which show that neither the *Implicit Euler* nor the *Classical Runge-Kutta* methods are symmetric, while the opposite holds for the *Stoermer-Verlet* method.

6.2 Order of convergence

In chapter 4.1 it was already assumed that both the *Explicit Euler* as well as the *Implicit Euler* methods fulfill $p = 1$. The order of convergence is an important characteristic of a numerical ODE solver since it directly correlates with the ability of a method to yield solution curves close to the analytical one. As the final task in this report, the order of convergence of the *Explicit Euler* method will be proven and the ones of the *Stoermer-Verlet* and the *RadauIIA* methods will be investigated in the same way.

In a similar way as in chapter 1, the order of convergence can be obtained by

$$p = \frac{\log(\max \Delta E)}{\log(h)} \quad (6.4)$$

where ΔE is the error in energy for $t = nh$ with $n = 1, \dots, N$, while $N = \frac{T}{h}$. The set of parameters in (6.2) will be applied once again, only with $\omega = 1$. The analytical energy is calculated by (2.2) as the referential values. The set of parameters in (6.2) will be used once again. In order to read p of with ease, logarithmic plots for both h and ΔE will be used. There, p can be obtained by evaluating the slope of the resulting curve since.

6.2.1 Order of convergence of the Explicit Euler algorithm

The mentioned plot for the *Explicit Euler* method is obtained as follows:

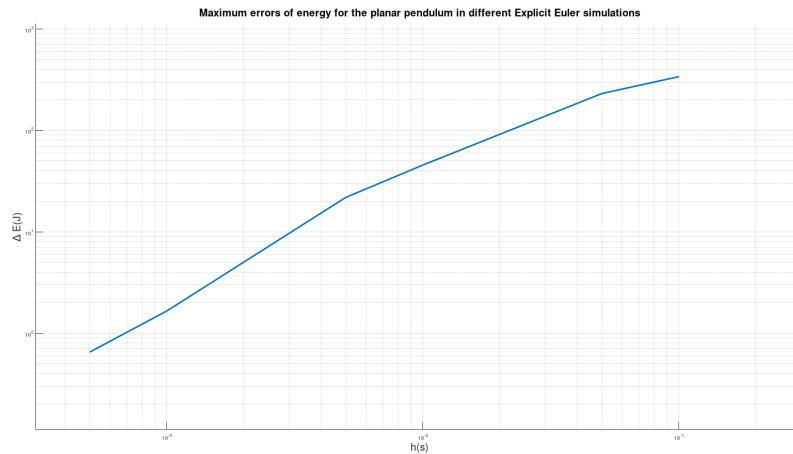


Figure 6.5: Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005, 0.001, 0.0005]$ after simulating with the Explicit Euler algorithm

While the p obviously differs for different h , the order is always close to 1 at least, while the assumption even seems to hold for certain h . In a way, it also makes sense that the order is lower for larger step sizes when looking at results obtained in 4.1.

6.2.2 Order of convergence of the Stoermer-Verlet algorithm

The same plot for the *Stoermer-Verlet* method renders as follows:

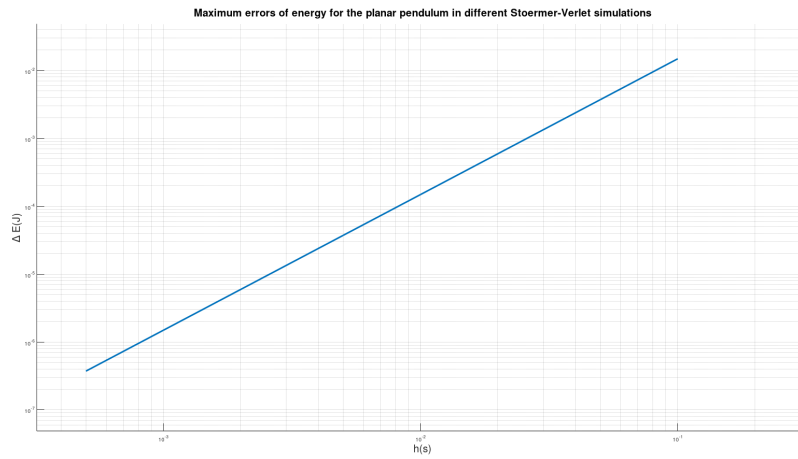


Figure 6.6: Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005, 0.001, 0.0005]$ after simulating with the Stoermer-Verlet algorithm

Again, one can read p as the slope, which is approximately 2. This is according to theory, as this symplectic method inherits two *Explicit Euler* methods.

6.2.3 Order of convergence of the RadauIIA algorithm

The plot for the *RadauIIA* method has been adopted, since the simulation was very time-consuming. Therefore, it was only conducted for $h = [0.1, 0.05, 0.01, 0.005]$ as it already gives a convenient impression regarding the order of convergence of the *RadauIIA* method:

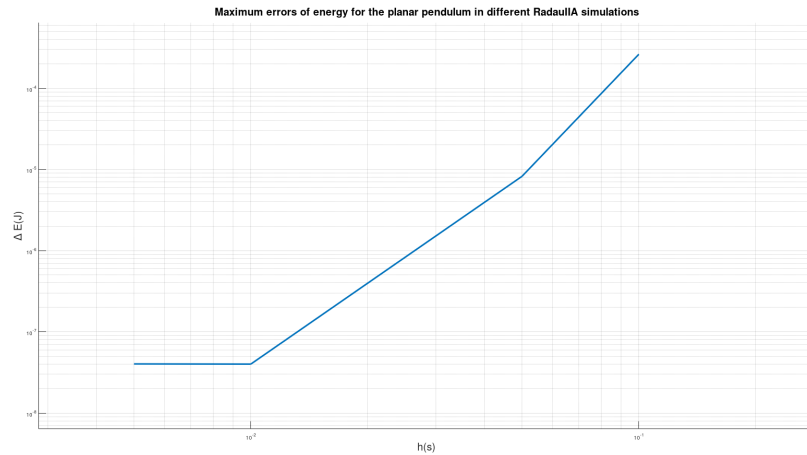


Figure 6.7: Maximum error in energy for each $h = [0.1, 0.05, 0.01, 0.005]$ after simulating with the RadauIIA algorithm

Looking at the plot on a macroscopic scale, it can be seen that $p \approx 4$, which matches theoretical results obtained during the course. For smaller h , the error converges to $10^{(-7)}$. This might happen because the referencial value for the analytically calculated energy is of limited accuracy in comparison to the very small errors of this method for small h .